# Filtered Approximate Nearest Neighbor Search on Vectors with Diverse Labels

## 1 PSEUDO CODE OF ApproximateELS

The approximated ELS retrieval method (ApproximateELS, Algorithm 1) initially checks whether $L_q \in F$ (Lines 2-6). If $L_q \in F$, it returns $\hat{F}_q = \{L_q\}$ (Line 8). Otherwise, the algorithm enumerates the nodes in $\mathbb{I}(l_{\max})$, where $l_{\max}$ is the queried label with the largest ID (Lines 9-10). Each such node is treated as a possible anchor on a trie path that may contain all labels in $L_q$. The function BottomUp-Traversal then walks from the anchor toward the root and matches query labels in reverse ID order (Lines 12-21). During this traversal, ancestor labels with IDs larger than the current target label are skipped, a matched label advances the query pointer, and the traversal is pruned once a smaller label or the root is reached before all query labels are matched (Lines 16–21). If all query labels are matched, the anchor is regarded as a valid starting point for relaxed ELSs (Lines 22-23).

Finally, CollectCandidateELSs performs a breadth-first traversal from the validated anchor node and inserts every encountered terminal label set into $\hat{F}_q^{(a)}$ (Lines 24-32). Note that, since each label set in $F$ corresponds to a unique terminal node in the trie and $\hat{F}_q^{(a)}$ is maintained as a set, repeated collections of the same label set, if any, are automatically removed by the insert operation. Unlike exact ELS retrieval, ApproximateELS does not further filter these candidates to retain only minimal supersets of $L_q$. Therefore, the returned set may contain non-minimal supersets, but each collected terminal label set remains compatible with the query-label containment condition.

**Time complexity of** ApproximateELS**.** Lines 1–8 check whether $L_q$ itself is a terminal label set in the trie tree, which takes $O(|L_q|)$ time. Then, Lines 9–10 enumerate all nodes in $\mathbb{I}(l_{\max})$, and Lines 13–21 perform a bottom-up traversal from each enumerated node. Since each trie path has length at most $|\mathcal{L}|$, this part costs

$$O\big(|\mathcal{L}| \cdot |\mathbb{I}(l_{\max})|\big). \tag{1}$$

For every validated anchor node, Lines 24–32 collect candidate ELSs by downward traversals. Let $\Upsilon$ denote the number of such downward traversal paths. Since each path costs at most $O(|\mathcal{L}|)$, this part costs

$$O\big(|\mathcal{L}| \cdot \Upsilon\big). \tag{2}$$

Therefore, the total time complexity of ApproximateELS is

$$O\Big(|\mathcal{L}| \cdot \big(|\mathbb{I}(l_{\max})| + \Upsilon\big)\Big), \tag{3}$$

where the $O(|L_q|)$ checking cost is dominated by the trie traversal cost. Compared with the exact ELS retrieval method, ApproximateELS skips the minimal-superset filtering step, and therefore has a smaller time complexity.

**Algorithm 1** ApproximateELS

**Input:** trie tree $T$ with root $r$, inverted list $\mathbb{I}$, and the queried label set $L_q$
**Output:** the set $\hat{F}_q^{(a)}$ of approximated ELSs

```
1:  F̂_q^(a) ← ∅, current node c ← root r of T
2:  for each queried label l ∈ L_q in the increasing order of IDs do
3:      if no child of c corresponds to l then
4:          break
5:      else
6:          c ← child of c that corresponds to l
7:  if c is a terminal node that corresponds to L_q then
8:      return F̂_q^(a) = {L_q}
9:  for each node u ∈ I(l_max) do
10:     BottomUp-Traversal(L_q, u)
11: return F̂_q^(a)
12: function BottomUp-Traversal(L_q, u)
13:     u_initial ← u, i ← |L_q| − 1
14:     while i ≥ 0 do
15:         u_p ← parent of u on the trie tree
16:         if u_p matches a label with a larger ID than L_q[i] then
17:             u ← u_p
18:         else if u_p matches L_q[i] then
19:             u ← u_p, i ← i − 1
20:         else if u_p matches a label with a smaller ID than L_q[i] or u_p is r then
21:             break
22:     if i < 0 then
23:         CollectCandidateELSs(u_initial)
24: function CollectCandidateELSs(u_i)
25:     Q ← {u_i}
26:     while Q ≠ ∅ do
27:         v ← Q.dequeue()
28:         if v is a terminal node corresponding to L ∈ F then
29:             F̂_q^(a).insert(L)
30:         else
31:             for each child x of v on T do
32:                 Q.enqueue(x)
```

## 2 THEOREM AND PROOF

Theorem 1. *For every label set $L \in \hat{F}_q^{(a)}$ such that $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}$, by employing at least one entry vector whose associated label set is $L$, the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones.*

Proof. First, for any $S \subseteq F$, we use $Desc^+(S)$ to represent the group of label sets that can be reached by a traversal of LNG starting from nodes that correspond to label sets inside $S$. Since $\hat{F}_q$ is the set of minimum supersets of $L_q$ on LNG, we have

$$Desc^+(\hat{F}_q) = F_{pass}. \tag{4}$$

Moreover, since $\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}$, we have

$$Desc^+(\hat{F}_q) \subseteq Desc^+(\hat{F}_q^{(a)}) \subseteq Desc^+(F_{pass}). \tag{5}$$

Furthermore, given that edges on LNG indicate label set containment relations, we have

$$Desc^+(F_{pass}) = F_{pass}. \tag{6}$$

The above equations indicate that

$$Desc^+(\hat{F}_q^{(a)}) = F_{pass}. \tag{7}$$

That is to say, a traversal of LNG starting from nodes that correspond to label sets inside $\hat{F}_q^{(a)}$ reaches the exact set of qualified label sets. Further consider a traversal of the unified graph starting from at least one entry vector whose associated label set is $L$. The combination of cross-group and intra-group proximity edges enables this traversal to exactly reach all vectors whose associated label set is inside $Desc^+(\{L\})$. As a result, Equation (7) indicates that, the traversal on the unified graph can reach all qualified vectors without encountering any unqualified ones, by employing at least

one entry vector whose associated label set is $L$, for every $L \in \hat{F}_q^{(a)}$. Hence, this theorem holds. □

Theorem 2. ApproximateELS *returns a set* $\hat{F}_q^{(a)}$ *of qualified label sets such that*

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}, \tag{8}$$

*where* $F_{pass} = \{L \in F \mid L_q \subseteq L\}$.

Proof. We prove the theorem by showing the two containment relations.

First, we show that $\hat{F}_q \subseteq \hat{F}_q^{(a)}$. If $L_q \in F$, then $L_q$ itself is the only minimum superset of $L_q$ in $F$. In this case, both the exact ELS retrieval method and ApproximateELS return $\{L_q\}$, and thus the claim holds. Otherwise, when $L_q \notin F$, the exact ELS retrieval method first enumerates candidate label sets whose corresponding trie paths contain all labels in $L_q$, and then removes non-minimal supersets to obtain the exact ELS set $\hat{F}_q$. In contrast, ApproximateELS keeps these candidate label sets without applying the minimality filtering step. Therefore, every exact ELS is retained by ApproximateELS, which gives

$$\hat{F}_q \subseteq \hat{F}_q^{(a)}. \tag{9}$$

Second, we show that $\hat{F}_q^{(a)} \subseteq F_{pass}$. For any $L \in \hat{F}_q^{(a)}$, ApproximateELS inserts $L$ only after the corresponding trie path has been verified to contain all labels in $L_q$. Since each terminal node in the trie corresponds to a unique label set in $F$, this implies

$$L_q \subseteq L. \tag{10}$$

By the definition of $F_{pass}$, we have $L \in F_{pass}$. Hence,

$$\hat{F}_q^{(a)} \subseteq F_{pass}. \tag{11}$$

Combining Equations 9 and 11, we obtain

$$\hat{F}_q \subseteq \hat{F}_q^{(a)} \subseteq F_{pass}. \tag{12}$$

Thus, the theorem holds. □

## 3 XGBOOST ROUTER TRAINING

In Sections 4.1 and 4.2, we theoretically analyze the strengths and limitations of existing state-of-the-art methods. Based on these analyses, we propose SODA, which intelligently and adaptively selects among three search paradigms in real time: unified-graph-based search, proximity-graph-based search, and pre-filtering brute-force search. By routing each query to a suitable paradigm, SODA avoids the limitations of any single method. We further select appropriate features and employ a lightweight learning model, specifically XGBoost, as a low-latency implementation of the structure-aware routing function. XGBoost is a gradient-boosted decision tree ensemble that constructs trees sequentially, where each new tree corrects the residual errors of previous ones. It can capture nonlinear feature interactions with low inference latency, making it suitable for the routing task. Specifically, we set the maximum depth of each tree to 6, which provides high classification accuracy among the three search paradigms while avoiding overfitting. We set the learning rate to 0.5 and the number of boosting rounds to 300.

We train a single XGBoost-based routing model on random LANNS tasks generated from all eight datasets, excluding the test queries used in our main evaluation. In total, we generate 40,000 training queries, with 5,000 queries from each dataset, and train XGBoost on CPUs. As shown in Table 8, the training time is typically only a few seconds, which is negligible compared with the overall experimental cost. For each query, we choose the search paradigm with the highest QPS among those whose recall is above 0.9, and use its paradigm label as the training target.

For evaluation, we split the queries of each dataset into 80% for training and 20% for testing. That is, the model is trained on the combined training queries from all eight datasets, and then evaluated separately on the test queries of each dataset. We also conduct a leave-one-dataset-out experiment, where all queries from one dataset are removed from the training set and the trained model is evaluated on the test queries of the held-out dataset. As shown in Tables 8 and 9, the routing accuracy remains above 90% in both settings, indicating the generality of the routing model. Note that, prediction accuracy is conservative because a misprediction may still select a paradigm with similar QPS. Together with Figure 11 and the end-to-end results in Figures 6–10, the high leave-one-dataset-out accuracy in Table 9 indicates that harmful misrouting is infrequent.

Finally, in all experiments that evaluate algorithm performance, we report the end-to-end latency from feeding vectors and labels into the algorithm to returning the top-$K$ results. This latency includes both feature extraction and model prediction. As shown in Table 8, this overhead is typically only a few microseconds, indicating that our routing framework can make real-time predictions.